dlnmpy: distributed lag non-linear models in Python, checked against R and against the authors' published code

Craig Parker Geir [CHECK: surname and ORCID]
Benjamin de Vantier du Plessis [CHECK: spelling of surname, ORCID]
Sam Zermatten-Jack [CHECK: spelling of surname, ORCID]
Sibusisiwe Audrey Makhanya [CHECK: you said Sibusiso; ORCID]
Nicholas Brian Brink [CHECK: joins only if he approves the manuscript; ORCID]
Admire Chikandiwa [CHECK: ORCID] Matthew Chersich [CHECK: ORCID]

Abstract

Distributed lag non-linear models (DLNMs) are the standard framework for exposure-lag-response associations in environmental epidemiology, especially temperature and mortality. The reference implementation is the R package dlnm, with further R packages and scripts for the quantities applied papers report. This paper presents dlnmpy, a Python implementation of the framework and its workflow with no R dependency. It ports dlnm, with R's spline, pretty() and quantile() routines so that default knots match, and adds attributable risk, the minimum mortality temperature, penalised DLNMs and a one-call workflow. Meta-analysis is in the companion package mixmetapy. Agreement with R is checked in three layers: fixtures generated in R, five analyses run in both languages (98 comparisons, each within 1e-10 + 1e-8|r| of R's value r), and six published analyses whose authors' R code was translated into Python and rerun. For England and Wales, the code of Gasparrini et al. (2015) gives a median minimum mortality percentile of 89.5 and an attributable fraction of 8.94%, and that of Gasparrini and Armstrong (2013) gives pooled relative risks of 1.101 at 22 °C and 1.309 at 0 °C. dlnmpy gives the same values, and the six translations agree with that code in all 161,356 comparisons at stated tolerances. These cover daily time series and one cohort, with log links, unit weights and, for penalised models, mgcv's smoothing parameters. Development found 41 defects between releases 0.4.1 and 0.8.4, 28 of them silent, each recorded in a ledger. The executable manuscript reruns the examples and all six translations.

1 Introduction

Exposure-lag-response associations arise whenever the effect of an exposure on an outcome is spread over time. Distributed lag non-linear models (DLNMs) describe them with a cross-basis, the tensor product of a basis for the exposure-response relationship and a basis for the lag-response relationship (Gasparrini, Armstrong, and Kenward 2010; Gasparrini 2014). The framework is general. Its main use has been in time-series studies of ambient temperature and daily mortality, where it underpins the multi-country estimates of mortality attributable to heat and cold (Gasparrini et al. 2015) and the projections of those burdens under climate change (Vicedo-Cabrera, Sera, and Gasparrini 2019).

The method has one reference implementation, the R package dlnm (Gasparrini 2011; Gasparrini and Armstrong 2009). Around it sits a workflow the applied literature treats as standard: reduction of location-specific fits and pooling by multivariate meta-analysis (Gasparrini, Armstrong, and Kenward 2012; Gasparrini and Armstrong 2013; Sera et al. 2019), attributable numbers and fractions (Gasparrini and Leone 2014), the minimum mortality temperature with its interval (Tobías, Armstrong, and Gasparrini 2017), and penalised DLNMs fitted through

mgcv (Gasparrini et al. 2017). Some of this is R packages, some is R functions published alongside papers, and some is scripts.

All of it is in R. Climate and exposure data increasingly arrive as gridded fields from reanalyses, satellites and downscaled projections, and much of the tooling built to handle them, such as xarray (Hoyer and Hamman 2017), is in Python. So are many health-data platforms and machine-learning pipelines. A researcher working in Python who wants a DLNM can move the data into R, call R from Python, or write a cross-basis by hand. Calling R keeps R's numerics but adds a second language and runtime to install and maintain. A hand-written cross-basis carries none of the reduction, attribution or checks.

This paper presents dlnmpy (Parker 2026a), a Python package that implements the DLNM framework and the workflow built on it, and describes how it was checked. The package has three aims. The first is fidelity to R: each ported function reproduces its R counterpart, including R's spline, `pretty()` and `quantile()` routines, so the default knots and prediction grids match. The second is scope, so that a temperature-mortality analysis runs from the raw series to the usual figures without leaving Python. The third is that the checks should be rerunnable. The R-generated fixtures ship with the package, the mathematics is written down as a language-neutral specification, and published analyses are rerun from their authors' code.

The central claim of a port is numerical equivalence with the original, and the evidence for it is only as good as the reader's ability to rerun it. This manuscript is therefore written as an executable document. The worked examples in Section 4 and the six published reproductions in Section 5 are rerun every time it is rendered.

Section 2 summarises the framework. Section 3 describes the design of the package, Section 4 illustrates it, and Section 5 reports the checks against R and what they found. Section 6 covers use with gridded and spatial data, Section 7 relates dlnmpy to other software, Section 8 sets out its limitations, and Section 9 describes how to reproduce the paper.

2 The framework and the workflow

This section fixes notation and names the components dlnmpy implements. The theory is in the papers cited, and `docs/theory.md` in the repository restates it as an implementation specification.

2.1 Cross-basis, prediction and reduction

Let x_t be an exposure series and y_t an outcome. A DLNM models the contribution of exposure history to the linear predictor as

$$s(x_t; \eta) = \sum_{l=\ell_0}^{L} f \cdot w(x_{t-l}, l),$$

where $f \cdot w$ is a bi-dimensional exposure-lag-response function. It is represented by a cross-basis matrix $\mathbf{W}$, built from v_x basis functions in the exposure dimension and v_l in the lag dimension, with a coefficient vector η of length $v_x v_l$ (Gasparrini 2014). Any regression model that accepts a design matrix can be used, and the later steps need only $\hat{\eta}$ and its covariance matrix.

Prediction is a linear map of $\hat{\eta}$. For a grid of exposure values and lags the cross-basis is rebuilt and multiplied by the coefficients. This gives lag-specific effects, the overall cumulative effect summed over lags, and lag-response curves at chosen exposures, each relative to a reference (centring) value and with standard errors from the covariance matrix. Reduction re-expresses

the fit in one dimension: as v_x coefficients for the overall cumulative exposure-response or for the exposure-response at one lag, or as v_l coefficients for the lag-response at one exposure value, each with its covariance. A location-specific fit then becomes a short vector suitable for pooling (Gasparrini and Armstrong 2013).

2.2 Two-stage designs, attribution and penalised models

In a multi-location study each location is fitted separately and reduced, and the reduced coefficients are pooled with a multivariate random-effects meta-analysis, optionally with location-level meta-predictors (Gasparrini, Armstrong, and Kenward 2012; Sera et al. 2019). Best linear unbiased predictions (BLUPs) give location-specific curves that borrow strength from the pool.

Attributable numbers and fractions are computed from the fitted exposure-response, from the backward or the forward perspective, over the whole exposure range or parts of it, with empirical intervals from simulated coefficients (Gasparrini and Leone 2014). The minimum mortality temperature (MMT) is the exposure at which the overall cumulative curve is lowest, searched within a restricted range of the exposure distribution (by default in dlnmpy, the 1st to 99th percentiles), with an interval by the same simulation (Tobías, Armstrong, and Gasparrini 2017). Penalised DLNMs start from a rich basis and control its smoothness with penalties on the cross-basis coefficients, which makes results less sensitive to the choice of knots. The smoothing parameters are selected by restricted maximum likelihood (Gasparrini et al. 2017; Wood 2011).

3 Design of the package

3.1 Principles

A port of the R code. Each function was written from the R source of the function it replaces and from the published methodology, and reproduces R's behaviour including its corner cases: how `crossbasis` fills the first L rows with missing values, how `crosspred` builds its default grid, how `glm` drops aliased columns. Where R accepts an input that it cannot later use, dlnmpy refuses at construction and says why. One example is a `ps` lag basis with too few degrees of freedom, which R builds and then cannot predict from. R argument spellings are accepted, including R's partial matching of `Boundary.knots` and `intercept` (`Bound=`, `int=`), so the published scripts of the dlnm authors translate almost line by line. The price is an API that looks like R: arguments passed as dictionaries, coefficients found by column-name prefix, and `from_` for R's `from`. Section 4.3 shows the route for users who prefer to work with plain arrays.

R's own numerical routines. dlnm's defaults depend on R's `pretty()`, on `quantile()` type 7 and on the splines package. dlnmpy ports these from the R sources. Substituting the NumPy equivalents would move default knots, because `numpy.percentile` and R's `quantile()` differ in small ways.

No R at run time. dlnmpy depends on NumPy, SciPy and pandas (Harris et al. 2020; Virtanen et al. 2020; McKinney 2010). statsmodels (Seabold and Perktold 2010) is optional, for fitting, and matplotlib (Hunter 2007) is optional, for plotting. The example datasets from dlnm ship with the package.

Fixtures as the contract. Numerical behaviour is pinned by fixtures generated in R and shipped in the repository. A change that alters a number is a failing test. The mathematics is written as a language-neutral specification with a porting guide, so the same fixtures can check an implementation in another language.

R's package boundaries. dlnmpy follows the division of the R ecosystem. The DLNM machinery, penalised fitting and the attribution functions that accompany dlnm are in dlnmpy. Multivariate

meta-analysis, which in R is the separate package mixmeta with its own methods paper and users who never fit a DLNM, is the companion package mixmetapy (Parker 2026b). The two were one package until release 0.8.0. mixmetapy takes coefficient and covariance matrices only and does not import dlnmpy, and `pip install "dlnmpy[twostage]"` installs both.

3.2 Modules and the mapping from R

Table 1 lists the R functions and their counterparts.

Table 1: Correspondence between R functions and the dlnmpy and mixmetapy API.

R	dlnmpy	Notes
`onebasis`, `crossbasis`	`onebasis`, `crossbasis`	exposure may be a series or a matrix of exposure histories
`lin`, `poly`, `strata`, `thr`, `integer`	same names	
`splines::ns`, `splines::bs`	`ns`, `bs`	ports, including R's extrapolation beyond the boundary knots
`ps`, `cr`, `cbPen`	`ps`, `cr`, `cbpen`	penalty matrices; `cr` ports the mgcv construction
`logknots`, `equalknots`, `exphist`	same names	
`crosspred`, `crossreduce`	`crosspred`, `crossreduce`	`from` becomes `from_`
`glm`, `survival::clogit`	`fit_glm`, `fit_clogit`	R's handling of aliased columns and missing rows
`mgcv::gam(paraPen=)`	`fit_pgam`, `fit_pglm`	penalised IRLS with REML or ML smoothing
`mgcv::gam` with `s(x, lag, bs = "cb")`	not ported	`crosspred` on these internal smooths is not implemented
`attrdl.R`, `findmin.R`	`attrdl`, `findmin`, `mmt`, `attr_table`	attribution module
`mixmeta`	`mixmetapy.mixmeta`	companion package, single-level models, with `stack_reduced` and `predict_reduced` in dlnmpy
`plot.crosspred`, `plot.crossreduce`	`.plot()` methods	matplotlib
`chicagoNMMAPS`, `drug`, `nested`	`dlnmpy.datasets`	

`onebasis` and `crossbasis` return objects that carry the attributes R stores on its matrices (the basis function, its arguments, knots, boundary knots, lag range and group structure), which `crosspred` and `crossreduce` need to rebuild the basis at prediction. The cross-basis columns are named `v{i}.l{j}` as in R.

`crosspred` and `crossreduce` need only the coefficients and covariance of the cross-basis columns, so any fitting library can be used, and the two arrays can be passed directly with `coef=` and `vcov=`. For equivalence with R, `fit_glm` wraps statsmodels and reproduces `glm` on rank-deficient designs. Aliased columns are detected with the criterion of LINPACK's `dqrdc2` and dropped, the IRLS step is solved by QR, and the quasi-likelihood dispersion uses R's residual degrees of

freedom. `fit_clogit` fits the conditional logistic model for case-crossover and matched designs by Newton-Raphson on the exact conditional likelihood, which is `survival::clogit`'s default, and a step-halving Newton, as in `survival::coxph`, takes over when the plain Newton step overflows. `fit_pgam` and `fit_pglm` fit penalised models by penalised IRLS with the smoothing parameters chosen by REML or ML.

`attrdl` and `findmin` port the published functions with the same arguments. `mmt` and `attr_table` wrap them into the summaries used in practice, with one shared set of simulated coefficients so that the cold, heat, extreme and moderate components are mutually consistent. `attrdl` also accepts a `group` argument for stacked series, which the reference function does not have. The lagged exposures are then formed within each series.

Beyond the port, `dlnm()` fits the usual temperature-mortality model in one call and returns an object bound to its data, whose methods give the MMT, relative risks at chosen percentiles, the attributable table and the standard figure. It is tested to agree with the explicit pipeline to 10^{-12}.

Three behaviours inherited from R need care in use. Predictions are relative to a centring value, and when none is given dlnmpy chooses one and raises a `CenteringWarning`, as R prints a message. Natural and B-splines extrapolate beyond their boundary knots, as in R, so predictions outside the observed exposure range rest on the basis and not on data. The backward and forward attribution perspectives answer different questions. The backward perspective attributes the cases on day t to the exposures over the preceding lags, and the forward perspective attributes to the exposure on day t the cases over the following lags. Both treat the contributions at different lags as independent, and the forward version approximates the lag structure by averaging the future counts, which is expected to underestimate the attributable number slightly relative to the backward version (Gasparrini and Leone 2014).

3.3 Dependencies and distribution

dlnmpy requires Python 3.9 or later and is on PyPI. The repository contains GitHub Actions workflows that run the tests on Linux and macOS with Python 3.9 to 3.13, check a minimal installation and build the package, but they do not currently run on GitHub's hosted service. The test suite is therefore run locally before each release, and each release is archived on Zenodo. dlnmpy is licensed GPL-2.0-or-later as a derived work of dlnm and of R's splines and base packages, which means that software distributed with dlnmpy built into it must also be released under a compatible licence. Using it to analyse data carries no such condition.

4 Illustration

The examples use the `chicagoNMMAPS` data distributed with dlnm: daily deaths and temperature in Chicago from 1987 to 2000, 5,114 days.

4.1 One call from data to results

```
chicago = dl.datasets.chicago_nmmaps()

fit = dl.dlnm(chicago, outcome="death", exposure="temp", lag=21,
              argvar={"fun": "bs", "degree": 2,
                      "knots": dl.percentile_knots(chicago.temp, [10, 75,
                       ↪  90])},
              arglag={"fun": "ns", "knots": dl.logknots(21, 3)},
```

```
              time="time", df_per_year=7, dow="dow")
print(fit.summary())
```

```
DLNM: death ~ temp, lag 0-21, quasipoisson
observations: 5114 (fitted: 5093)   cross-basis df: 5 x 5
minimum-risk temp: 25.5 (95% CI 25.2 to 25.8), percentile 94.2 (2000 draws,
↪  seed 0)
RR at 1st percentile (-15.0): 1.335 (1.243, 1.433)
RR at 99th percentile (28.6): 1.193 (1.121, 1.269)
QAIC: 39383.0
```

The model is a quasi-Poisson regression of daily deaths on a cross-basis of temperature and lag, with a natural spline of time of 7 degrees of freedom per year and day of week. The temperature basis is a quadratic B-spline with knots at the 10th, 75th and 90th percentiles, and the lag basis a natural spline with log-spaced knots over lags 0 to 21. The temperature and lag bases are those Gasparrini et al. (2015) used in each location. Their seasonal spline had 8 degrees of freedom per year, and the 7 used here follow the dlnm vignette. The first 21 days have no complete lag history and are not fitted. `summary()` computes the MMT interval from 2,000 simulated coefficient sets with seed 0.

Table 2: Deaths attributable to temperature in Chicago, 1987 to 2000, relative to the minimum mortality temperature, with empirical 95% intervals from 5,000 simulated coefficient sets (seed 2026). Extreme means beyond the 2.5th or 97.5th percentile. The point estimates do not depend on the simulation. The interval limits change with the seed at this number of simulations, so each component's limits are rounded to the power of ten at or above their Monte Carlo standard deviation over ten seeds (see the text). As in the authors' published code, each extreme range and the moderate range next to it share their cut-off, and Chicago temperatures, converted from whole degrees Fahrenheit, fall exactly on it on some days, which then count in both.

Component	AN	AN 95% eCI	AF (%)	AF 95% eCI
total	85,037	62,000 to 107,000	14.41	10.5 to 18.1
cold	81,615	59,000 to 103,000	13.83	10.0 to 17.5
heat	3,564	2,400 to 4,700	0.60	0.41 to 0.79
extreme cold	4,403	3,300 to 5,500	0.75	0.56 to 0.93
moderate cold	77,874	56,000 to 99,000	13.19	9.4 to 16.8
moderate heat	310	130 to 490	0.05	0.022 to 0.083
extreme heat	3,380	2,300 to 4,400	0.57	0.40 to 0.75

```
ax = fit.figure(xlab="Temperature (°C)")
```

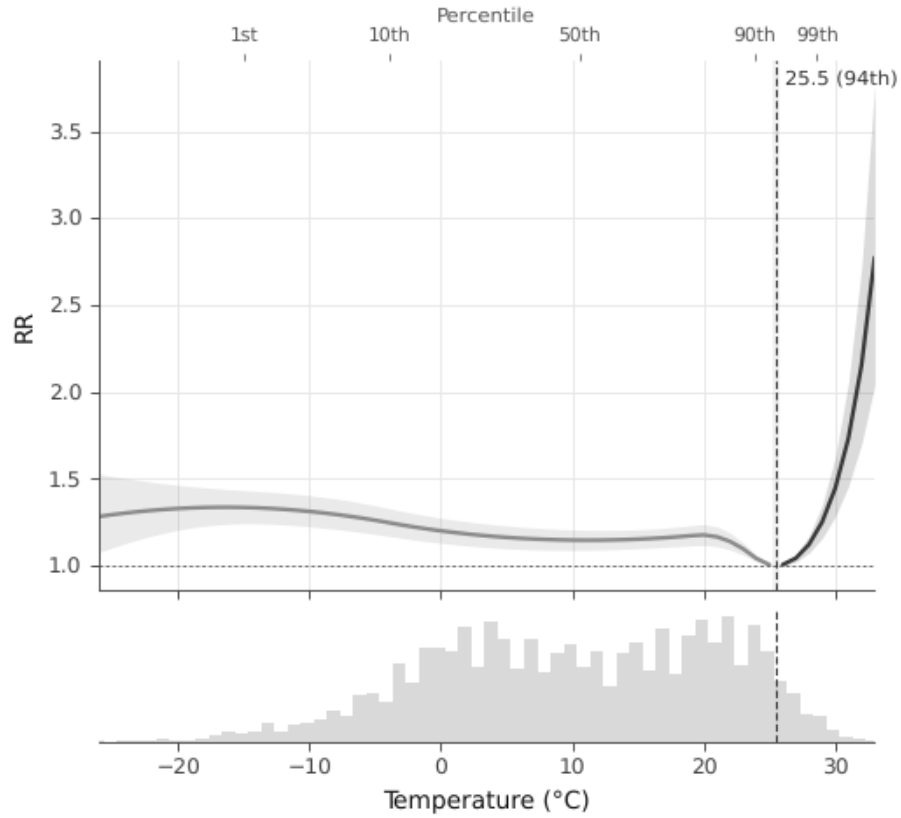


Figure 1: Overall cumulative association between daily mean temperature and mortality in Chicago, lags 0 to 21, from `fit.figure()`. The curve is centred on the minimum mortality temperature (vertical line, with its 95% empirical interval shaded). Percentiles of the temperature distribution are marked along the top and the distribution is shown beneath.

Over ten seeds (2026 to 2035) the standard deviation of the limits of the total attributable number was 396 deaths, and of the limits of the total fraction 0.07 percentage points. For any component it was at most 485 deaths. The point estimates do not depend on the simulation, but the limits cannot be read more finely than this.

Cold accounts for most of the temperature-attributable deaths in Chicago, and moderate cold for most of that, the same pattern Gasparrini et al. (2015) reported for most locations. `fit.predict(percentiles=[1, 99]).to_dataframe()` returns the lag-specific effects as a long table, and `fit.reduce()` returns the reduced coefficients for pooling.

4.2 The explicit pipeline

Every step is also available on its own, with the same names and arguments as in R.

```
cb = dl.crossbasis(chicago.temp, lag=21,
                   argvar={"fun": "ns", "df": 4},
                   arglag={"fun": "ns", "df": 4})
ns_time = dl.onebasis(chicago.time, "ns", df=7 * 14)
X = dl.design_matrix(chicago, ("cb", cb), ("ns_time", ns_time),
 ↪  intercept=False)
model = dl.fit_glm("death ~ " + " + ".join(X.columns) + " + C(dow)",
                   chicago.join(X), family="quasipoisson")
```

```
pred = dl.crosspred(cb, model, cen=21, by=1, name="cb")   # centred at 21 °C
red = dl.crossreduce(cb, model, cen=21, name="cb")         # overall
 ↪  cumulative, for pooling
print(pred.overall().query("var in [-20, 0,
 ↪  33]").round(3).to_string(index=False))
```

```
  var   fit   low  high
-20.0 1.329 1.225 1.443
  0.0 1.147 1.085 1.213
 33.0 0.922 0.827 1.028
```

4.3 Working with plain arrays

`crosspred` finds the cross-basis coefficients in a fitted model by the `name` prefix, as dlnm does. A user who prefers to build the design matrix and fit the model in their own way can instead pass the coefficients, their covariance and the link. The fit below uses statsmodels directly on NumPy arrays. The printed number is the largest difference between its predicted relative risks and those of the pipeline above.

```
import statsmodels.api as sm

W = np.asarray(cb)                                          # cross-basis as a
 ↪  plain array
T = np.asarray(ns_time)
D = np.eye(7)[chicago.dow.astype("category").cat.codes][:, 1:]
Xd = np.column_stack([np.ones(len(chicago)), W, T, D])
ok = ~np.isnan(Xd).any(axis=1)                              # first 21 days
 ↪  have no lag history
res = sm.GLM(chicago.death.to_numpy()[ok], Xd[ok],
 ↪  family=sm.families.Poisson()).fit(scale="X2")

k = slice(1, 1 + W.shape[1])                                # the cross-basis
 ↪  columns
pred2 = dl.crosspred(cb, coef=res.params[k], vcov=res.cov_params()[k, k],
                     model_link="log", cen=21, by=1)
print(f"{np.max(np.abs(pred2.allRRfit - pred.allRRfit)):.1e}")
```

```
4.7e-14
```

4.4 A two-stage analysis

A multi-location study fits each location, reduces it, pools with mixmetapy and predicts back through the basis. The example below uses twelve locations simulated with `dl.datasets.simulate_cities()`, with location mean temperature as a meta-predictor. Section 6 repeats the design on gridded data.

```
from mixmetapy import mixmeta

sim = dl.datasets.simulate_cities(n_cities=12, n_days=2000, seed=1)
knots, bk, cen = [8, 15, 22], [-5, 40], 18
reduced, cbs, mean_temp = [], [], []
for city, d in sim.groupby("city"):
```

```
    d = d.reset_index(drop=True)
    cb_i = dl.crossbasis(d.tmean, lag=10,
                         argvar={"fun": "ns", "knots": knots, "boundary_knots":
 ↪  bk},
                         arglag={"fun": "ns", "df": 3})
    X_i = dl.design_matrix(d, ("cb", cb_i),
                           ("ns_time", dl.onebasis(d.time, "ns", df=18)),
 ↪  intercept=False)
    fit_i = dl.fit_glm("y ~ " + " + ".join(X_i.columns), d.join(X_i),
 ↪  family="quasipoisson")
    reduced.append(dl.crossreduce(cb_i, fit_i, cen=cen, name="cb"))
    cbs.append(cb_i)
    mean_temp.append(d.tmean.mean())

y, S = dl.stack_reduced(reduced)
mm = mixmeta(y, S, method="reml")
mr = mixmeta(y, S, X=np.column_stack([np.ones(len(y)), mean_temp]),
 ↪  method="reml")
pooled = dl.predict_reduced(cbs[0], mm.coef_vec, mm.vcov, at=np.arange(-5, 41),
 ↪  cen=cen)
q0, q1 = mm.qtest(), mr.qtest()
print(f"I2 {q0['I2'][0]:.1f}% without and {q1['I2'][0]:.1f}% with the
 ↪  meta-predictor; "
      f"AIC {mm.aic:.2f} and {mr.aic:.2f}")
```

```
I2 44.9% without and 13.0% with the meta-predictor; AIC -9.08 and -10.36
```

5 Checks against R

The central claim of the package is numerical equivalence with R. It rests on three layers, each catching a class of error the previous one does not. One limit on their independence applies throughout. Apart from the calls into R packages and the authors' own published scripts, the R code used for comparison (the fixture generators and the side-by-side analyses) was written in the same development process as the port, so a shared misreading of an R argument would pass the first two layers. The third layer runs analysis code written independently of this project, but the R scripts that drive it and store its results, the translations, and the tolerances they apply came from the same process as the port. What is independent is the authors' analysis code and the numbers it produces.

5.1 Unit fixtures

`tools/make_fixtures.R` and its companions run dlnm, splines, mgcv, mixmeta, survival and the published `attrdl.R` and `findmin.R` on the vignette examples, and write the intermediate objects to `tests/fixtures/`: basis matrices for 36 basis specifications with and without values outside the fitting range, 12 cross-bases, 12 prediction objects, 7 reductions, the penalty matrices, and R's `pretty()`, `quantile()` and knot helpers. The test suite (170 tests) compares against them. Tolerances are 10^{-10} to 10^{-12} for the bases and predictions, 10^{-6} to 10^{-8} for attribution and QAIC, and looser for the penalised fits against mgcv, which Section 8 discusses. `fit_clogit`, `dlnm()`, the simulation helpers in `dlnmpy.uncertainty` (named `bootstrap`, though they simulate the coefficients and do not resample the data) and plotting have no R fixture of their own.

5.2 Side-by-side analyses

`tools/side_by_side.R` and `tools/side_by_side.py` run five complete analyses in both languages, including the model fit, on data and specifications not used for the fixtures. They cover ordinary least squares on exposure histories in the `drug` trial, conditional logistic regression in the `nested` case-control study, logistic and quasi-Poisson Chicago models with threshold, polynomial, strata and integer bases, and a four-city simulation with known truth. The R results are stored in the repository, so the Python half reruns without R. Each row compares a vector or matrix (coefficients, covariance, a prediction) and passes if every element is within $10^{-10} + 10^{-8}|r|$ of the R value r, or a tighter bound for the bases. For the four simulated cities a further row reports the share of the prediction grid at which the 95% band of that one fit contains the true curve. It is a sanity check, not an estimate of coverage, and does not involve R.

```
102 comparisons, 0 failures
```

Of these 102 rows, 4 are sanity checks against the simulated truth, so 98 are comparisons with R, with 0 failures.

Audits at releases 0.5.0, 0.6.0 and 0.7.0 compared further edge cases and the workflow layer against R, with every quantity agreeing to 10^{-6} or better and most to 10^{-12}. The results are recorded in `CHANGELOG.md` and `docs/validation.md`. The scripts for the 0.6.0 and 0.7.0 comparisons are not in the repository, so those results cannot be rerun, and none of the evidence in the rest of this section depends on them.

5.3 Published analyses rerun from the authors' code

The authors of the methods behind the workflow published their R code and data for several papers. These scripts were written independently of dlnmpy, which makes them the strongest check available. Six were translated into Python, and all six are rerun below as part of this document. For each, the data are downloaded from a pinned commit of the authors' repository and checked against a SHA-256 hash, the R side is produced by a script in `tools/` that runs the authors' code and stores its results in `tests/reference/`, and the Python translation in `examples/` compares every quantity with those stored results. The first two use the ten regions of England and Wales, 1993 to 2006, which are the fully public part of the data behind them.

5.3.1 Gasparrini et al. (2015), *The Lancet*

`examples/lancet_2015.py` translates the five stages of the authors' published script: a DLNM per region, reduction to the overall cumulative curve, multivariate meta-regression with BLUPs, the MMT, and attributable deaths with empirical intervals. The paper's own estimates come from 384 locations in 13 countries and are not reproduced here. What is reproduced is the authors' public example script for England and Wales, and the figures in square brackets are that script's results. The minimum mortality percentile is the median of the ten regional values.

```
England & Wales, 1993-2006, 7,573,716 deaths
minimum-mortality percentile (median over regions): 89.5     [published code:
 ↪  89.5]

attributable fraction (%)        estimate      95% empirical CI
 ↪  [published code]
  total                           8.94         8.40 to  9.49          [8.94
   ↪  (8.40-9.48)]
  cold                            8.63         8.10 to  9.18          [8.63
   ↪  (8.08-9.19)]
```

```
heat                            0.31          0.27 to  0.35          [0.31
↪  (0.27-0.35)]
```

The point estimates agree with the R script to the printed digit. The intervals differ in the second decimal place because this script draws its own coefficient samples (1,000 per region, fixed seeds). A second script, `examples/lancet2015_vs_r.py`, compares every stage with R, replaying R's own normal deviates and eigenvectors so that the intervals are compared without Monte Carlo error.

```
lancet2015_vs_r: 2545 comparisons with R, max abs difference 1.01e+00, 0
↪  failures
```

Table 3: Agreement with R at each stage of the reproduction of Gasparrini et al. (2015), against the authors' code with the first-stage `glm` run to convergence, as computed in this render. Differences are absolute; the meta-regression rows in brackets are against R's mixmeta run to its optimum.

Stage	Quantity	Maximum absolute difference from R
First stage	reduced coefficients	5.9×10^{-14}
First stage	reduced covariance	5.2×10^{-12}
Meta-regression	log-likelihood	1.4×10^{-8}
Meta-regression	coefficients	1.1×10^{-5} (1.1×10^{-6})
Meta-regression	between-study covariance	3.0×10^{-6} (1.2×10^{-7})
MMT	minimum mortality percentile	0
MMT	minimum mortality temperature (°C)	4.3×10^{-14}
Attribution	attributable deaths, per region and total	0.21 deaths (1.4×10^{-5} relative)
Attribution	attributable fraction (percentage points)	3.1×10^{-5}

The R script stores three versions of the R side: the authors' code as published, the same code with `glm.control(epsilon = 1e-14)`, and the second stage refitted with mixmeta's optimiser tolerance at 10^{-14}. Against the published code, the first-stage coefficients differ by 4.8×10^{-10} and their covariance by 1.9×10^{-6} relative, which is R's default convergence tolerance acting mainly through the dispersion. The meta-regression estimates 30 parameters from 10 regions and its likelihood is flat, so R's default stopping rule leaves its coefficients about 10^{-5} from the optimum, and the last digits of any optimiser's answer depend on the platform. mvmeta, used in the published script, and mixmeta differ from each other by 5.0×10^{-6} at their defaults. dlnmpy's log-likelihood is 1.3×10^{-8} higher than R's at its default, and its coefficients are 1.1×10^{-6} from R's at the optimum.

5.3.2 Gasparrini and Armstrong (2013), *BMC Medical Research Methodology*

This paper introduced the reduction and meta-analysis of DLNM estimates. Its analysis exercises what the Lancet analysis does not: `crossreduce` in its overall and predictor-specific forms, two simpler moving-average lag models compared with the full model, random-effects pooling, Cochran's Q and I^2, meta-regression on latitude, and prediction from pooled coefficients. `examples/bmcmrm_2013.py` translates the authors' public script, which they updated in

December 2017 for later versions of dlnm and mvmeta. The figures in square brackets are the updated script's results.

```
England & Wales, 10 regions, 1993-2006
  minimum-mortality temperature        17.1 C            [R script: 17.1]
  pooled RR at 22 C                    1.101 (1.079-1.124)   [R script: 1.101
   ↪  (1.079-1.124)]
  pooled RR at  0 C                    1.309 (1.245-1.376)   [R script: 1.309
   ↪  (1.245-1.376)]
  I2  overall 63.8%   at 22 C 16.4%   at 0 C 63.5%      [R script: 63.8 / 16.4
   ↪  / 63.5]

  QAIC, summed over regions (lower is better):
    B-spline of lag 0-21         402827.3   [R script: 402827.3]
    constant of lag 0-3          406212.0   [R script: 406212.0]
    constant of lag 0-21         405966.3   [R script: 405966.3]
    -> the flexible lag model wins by 3139 QAIC units

  meta-regression on latitude: 2 predictors x 4 outcomes, logLik 47.3137
  pooled lag-response at 22 C: peak RR 1.1142 at lag 0.0
  pooled lag-response at  0 C: peak RR 1.0585 at lag 1.8
```

The updated script's results differ slightly from the values printed in the paper (Gasparrini and Armstrong 2013), which gives relative risks of 1.101 (1.078 to 1.124) at 22 °C and 1.308 (1.245 to 1.375) at 0 °C, and I^2 of 63.7%, 16.0% and 63.4%. dlnmpy agrees with the updated script, which was rerun for this paper with R 4.3.3, dlnm 2.4.7 and mvmeta 1.0.3 and gave the same relative risks, I^2 values and QAIC totals as those printed above. `examples/bmcmrm2013_vs_r.py` compares every intermediate with R: the five reductions in all ten regions, the pooled and fixed-effects fits, Q statistics, the meta-regression on latitude and its Wald tests, the twelve pooled curves and the QAIC of each region.

```
bmcmrm2013_vs_r: 22771 comparisons with R, max abs difference 1.77e-02, 0
 ↪  failures
```

Against the authors' code run to convergence, the reductions agree to 10^{-13} and their covariances to 2×10^{-11}, the QAIC totals to 10^{-6}, and the pooled relative risks to 2×10^{-6}. The largest absolute difference printed above is on the QAIC totals of the published fit, about 400,000 in size, and comes from R's default `glm` tolerance. Against R's meta-analyses run to their optimum, the pooled coefficients agree to 6×10^{-10} and the meta-regression coefficients to 3×10^{-7}. At its defaults R stops slightly short of the optimum of the pooled fit at 22 °C, with a log-likelihood of 168.244063 where the optimum, reached by both R at a tight tolerance and dlnmpy, is 168.244068.

5.3.3 Four further papers

The other four translations cover designs the first two do not. The exposure-lag-response paper (Gasparrini 2014) applies DLNMs to a cohort, the Colorado Plateau uranium miners, with radon exposure histories at lags of 2 to 40 years entering Cox models fitted on sampled risk sets. The attributable-risk paper (Gasparrini and Leone 2014) computes attributable numbers and fractions for London by both perspectives, for cold, heat and their extreme and mild parts. The tutorial of Vicedo-Cabrera, Sera, and Gasparrini (2019) projects temperature-attributable mortality under climate scenarios, with bias correction of the climate models. The penalised DLNM paper (Gasparrini et al. 2017) fits penalised models to the London series and to the miners cohort with mgcv. Table 4 summarises the comparisons, which are rerun below.

```
statmed2014: 102224 comparisons with R, max abs difference 3.7e-05, 0 failures
leone2014: 22690 comparisons with R, max abs difference 1.61e-01, 0 failures
vicedo2019: 11060 comparisons with R, max abs difference 5.4e-07, 0 failures
biomet2017: 66 comparisons with R, max abs difference 3.0e-07, 0 failures
```

Table 4: The four further reproductions. Tolerances are stated in each script, and differences are absolute unless marked relative.

Paper	What is compared	Agreement
Gasparrini (2014)	risk sets and exposure histories, cross-bases, the eight models of Table II (coefficients, covariance, log-likelihood, AIC, BIC, likelihood-ratio tests), Figures 1 to 3, Table III, the peak lag, the additional analyses and a reduction	coefficients and covariance to 10^{-9}, predictions within 10^{-6} of R's standard error, peak lags identical
Gasparrini and Leone (2014)	model, Tables 1 and 2 (attributable numbers and fractions, both perspectives, with intervals from 5,000 draws), daily attributable deaths, predictions	fractions to 10^{-13}, intervals from R's own draws to 4×10^{-6} relative
Vicedo-Cabrera, Sera, and Gasparrini (2019)	first-stage fits, MMT, bias-corrected series, simulated coefficients, projected attributable numbers and fractions by decade, model and scenario, with intervals	fractions to 10^{-12}, intervals to 10^{-10}
Gasparrini et al. (2017)	penalty matrices and ranks, REML score, coefficients, covariance, effective degrees of freedom and the overall curve, at mgcv's smoothing parameters	penalties to 10^{-15}, scores to 10^{-8}, coefficients to 4×10^{-10} standard errors for London and 10^{-7} for the miners

Three of these comparisons depend on how R's own fits stop. R's `glm` stops at a relative change of 10^{-8} in the deviance and `coxph` at 10^{-9} in the log-likelihood, a few iterations before the maximum. For the London model of Gasparrini and Leone (2014) this leaves the authors' coefficients 1.5×10^{-8} short in relative terms, and for model 8 of Gasparrini (2014) 1.6×10^{-6} standard errors short. The R scripts therefore store each fit twice, as the authors' code leaves it and refitted to convergence. The tightest comparisons are against the converged fits, and the distance to the authors' fits is printed alongside. For Gasparrini et al. (2017) the comparison is made at mgcv's smoothing parameters. When dlnmpy selects its own, its REML score was never worse than mgcv's. On one London model the first smoothing parameter went to a value near zero against mgcv's 1.15 with a score 1.0e-03 better, because the score is almost flat in that direction once the other two lag penalties are present. The other smoothing parameters agreed with mgcv's to within 0.2%. The published values also hold where the paper reports them. The projected change in the heat-attributable fraction from 2010 to 2019 to 2090 to 2099

under RCP8.5 in Vicedo-Cabrera, Sera, and Gasparrini (2019) is 3.95 percentage points (0.69 to 6.75), which the paper prints as 4.0 (0.7 to 6.8).

The translations depart from the authors' code in a few documented places, all listed at the top of each script. The internal `s(x, lag, bs = "cb")` smooths of Gasparrini et al. (2017) and the simulation studies of Gasparrini (2014) and Gasparrini et al. (2017) are not reproduced. The miners model of Gasparrini et al. (2017) uses mgcv's default UBRE criterion, which dlnmpy does not implement, so a REML fit of the same model was added for the comparison of smoothing-parameter selection.

5.4 What the checks cover

Table 5: Which layer of the checks against R calls each function: the unit tests (which compare with R fixtures, or with the stored results of the published analyses), the side-by-side analyses, and the six translated published analyses. Read from the source of the tests and scripts when this copy was rendered.

Function	Unit tests	Side by side	Published analyses
`onebasis`	yes	yes	yes
`crossbasis`	yes	yes	yes
`crosspred`	yes	yes	yes
`crossreduce`	yes	yes	yes
`fit_glm`	yes	yes	yes
`fit_clogit`	yes	yes	yes
`fit_pgam`	yes		
`cbpen`	yes		yes
`attrdl`	yes		yes
`attr_table`	yes		
`findmin`	yes		yes
`mmt`	yes		
`simulate_coef`	yes		yes
`qaic`	yes		yes
`mixmeta`	yes		yes
`dlnm`	yes		

Five of the six scripts count comparisons element by element, so a covariance matrix of 20 coefficients contributes 400 and a prediction grid one per point. The script for Gasparrini et al. (2017) counts each compared quantity once, so its 66 comparisons cover 142,275 elements. The counts show the size of each check. Its breadth is in Table 5 and in the list of quantities at the top of each script. Equivalence is shown for the configurations the published analyses use, which are daily time series and one cohort with sampled risk sets, log links, unit prior weights, and penalised fits at mgcv's smoothing parameters. Prior weights, attribution with a logit link, `group=` with forward attribution, and smoothing parameters selected by dlnmpy are checked against R only through the unit tests of their parts, or not at all.

5.5 What the comparisons found

The authors' 2013 script computes the quasi-likelihood information criterion (QAIC) of its three lag specifications (a flexible spline over lags 0 to 21 and moving averages over lags 0 to 3 and 0 to 21), although the paper compares them graphically. Before release 0.4.1, dlnmpy took the log-likelihood from statsmodels, which for a quasi-Poisson family divides the Poisson log-likelihood

by the estimated dispersion. The dispersion differs between candidate models, so the error does not cancel in a comparison, and it favours the more over-dispersed, less well-fitting specification. The old criterion ranked the lag 0 to 3 moving average first. The corrected criterion gives the ranking printed above, with the flexible model first, as the R script does. The unit test for the criterion had passed because it compared the implementation with itself. The defect was fixed in 0.4.1 with fixtures from R, and the 2013 translation, added in 0.5.0, showed what it would have done to a model choice made by QAIC.

Later comparisons found defects of the same kind. Translating Gasparrini (2014) showed that `fit_clogit` returned coefficients that were all missing, without raising an error, on designs with large columns such as a cross-basis of cumulative radon exposure, and that its standard errors could be wrong by up to 5×10^{-3} when the coefficients were very small. Translating Gasparrini et al. (2017) showed that the penalised fitter could return an all-missing fit while reporting convergence. Committing these translations for this paper found three more. `simulate_coef` claimed to reproduce R's simulated coefficients from R's normal deviates, which fails whenever the two languages' eigenvalue routines choose opposite signs for an eigenvector. In the London model of Gasparrini and Leone (2014) 8 of 20 had opposite signs, and the attributable-number intervals differed by up to 1.2%. `fit_clogit` lost its parameter names on the step-halving path. `int=`, the authors' spelling of `intercept`, was rejected. The audits found defects in the ported functions themselves. `crossbasis` discarded a centring value given in `argvar`, which R keeps, giving a relative risk of 1.343 where R gives 1.166. The link detection treated a complementary log-log link as a log link. `strata` accepted tied breaks that R rejects. Others sat in the Python layers around the port, for example an offset discarded on its way into the fitter and an exposure logged twice on a refit after an aliased column was dropped.

`docs/defects.md` lists every defect with the release that fixed it, the function, whether it was silent, how it was found and the regression test that guards it. A defect is counted there if it gave a wrong number, crashed, hung, or rejected input that R accepts. By that rule there were 41 between releases 0.4.1 and 0.8.4. Of these, 28 gave wrong results without an error and 13 crashed or refused valid input. By location, 16 were in functions ported from R and 25 in the Python layers around them. The audits against R found 28, the reproductions of published analyses 11, a user's run 1, and the source of 1 is not recorded. None was found by the unit fixtures, which is what they are for: they confirm that each piece is right once written, and the defects lay in how pieces were combined or in inputs the fixtures did not cover. The ledger names the 2 defects that have no targeted regression test. A statistical review of the package for this paper found 8 more, all silent and all in edge cases that no published analysis reached. They are fixed, with regression tests, for the next release and listed in the same ledger.

6 Use with gridded and spatial data

dlnmpy has no spatial model of its own. Spatial structure enters a DLNM analysis through the exposure assigned to each unit and through the second stage of a two-stage design, and both steps work with Python's gridded-data tools. The example below takes a gridded daily temperature field, forms population-weighted district means with xarray, fits a DLNM per district, pools the districts with a meta-regression on mean temperature, and returns district MMTs, relative risks and attributable fractions as a geopandas GeoDataFrame ready to map. The temperature field, population and death counts are all simulated.

```
import xarray as xr
import geopandas as gpd
from shapely.geometry import box
```

```
rng = np.random.default_rng(1)
lat, lon = np.linspace(-27, -25.75, 6), np.linspace(27.5, 28.25, 4)
days = pd.date_range("2010-01-01", periods=6 * 365, freq="D")
season = 18 + 6 * np.cos(2 * np.pi * (days.dayofyear.to_numpy() - 15) / 365)
t2m = xr.DataArray(season[:, None, None] + 1.5 * (lat[None, :, None] + 26.4)
                   + rng.normal(0, 3, (len(days), 6, 4)),
                   dims=("time", "lat", "lon"), coords={"time": days, "lat":
 ↪  lat, "lon": lon})
pop = xr.DataArray(rng.gamma(2, 1000, (6, 4)), dims=("lat", "lon"),
 ↪  coords={"lat": lat, "lon": lon})
district = xr.DataArray(np.repeat(np.arange(12).reshape(6, 2), 2, axis=1),
 ↪  name="district",
                        dims=("lat", "lon"), coords={"lat": lat, "lon": lon})

# population-weighted district means: a time x district table
tdist = ((t2m * pop).groupby(district).sum() /
 ↪  pop.groupby(district).sum()).to_pandas()

argvar = {"fun": "ns", "knots": [15, 20, 25], "boundary_knots": [0, 40]}
rows, reduced, bases, deaths = [], [], [], {}
for dist, temp in tdist.items():                        # one 1-D series per
 ↪  district
    d = pd.DataFrame({"tmean": temp.to_numpy(), "time": np.arange(len(temp)) +
 ↪  1})
    risk = np.convolve(0.002 * (d.tmean - 20) ** 2, np.ones(6) / 6,
 ↪  "full")[:len(d)]
    d["deaths"] = rng.poisson(30 * np.exp(risk))        # simulated death counts
    cb_d = dl.crossbasis(d.tmean, lag=5, argvar=argvar, arglag={"fun": "ns",
 ↪  "df": 3})
    X_d = dl.design_matrix(d, ("cb", cb_d), ("ns_time", dl.onebasis(d.time,
 ↪  "ns", df=42)),
                           intercept=False)
    fit_d = dl.fit_glm("deaths ~ " + " + ".join(X_d.columns), d.join(X_d),
 ↪  family="quasipoisson")
    reduced.append(dl.crossreduce(cb_d, fit_d, cen=20, name="cb"))
    bases.append(cb_d)
    deaths[dist] = d.deaths.to_numpy()
    rows.append({"district": dist, "tmean": temp.mean()})
meta = pd.DataFrame(rows)

y, S = dl.stack_reduced(reduced)                         # arrays, in the order
 ↪  of `meta`
mr = mixmeta(y, S, X=np.column_stack([np.ones(len(meta)), meta.tmean]),
 ↪  method="reml")
blup = mr.blup(se=True)

grid = dl.onebasis(np.arange(0, 40.1, 0.1), **argvar)   # the exposure basis,
 ↪  for the MMT search
for i, dist in enumerate(meta.district):
    lo, hi = np.round(np.percentile(tdist[dist], [1, 99]), 1)
    b, V = blup["blup"][i], blup["vcov"][i]
```

```
    meta.loc[i, "mmt"] = dl.findmin(grid, coef=b, vcov=V, from_=lo, to=hi,
    by=0.1)
    rr = dl.predict_reduced(bases[i], b, V, at=[30.0],
    cen=meta.mmt[i]).overall()
    meta.loc[i, "rr_30C"] = rr.fit.iloc[0]
    meta.loc[i, "af_pct"] = 100 * dl.attrdl(tdist[dist].to_numpy(), bases[i],
    deaths[dist],
                                            coef=b, vcov=V, model_link="log",
    type="af",
                                            dir="forw", cen=meta.mmt[i])

cells = gpd.GeoDataFrame(
    {"district": range(12)}, crs="EPSG:4326",
    geometry=[box(27.4 + 0.5 * (k % 2), -27.1 + 0.25 * (k // 2),
                  27.9 + 0.5 * (k % 2), -26.85 + 0.25 * (k // 2)) for k in
                   range(12)])
gdf = cells.merge(meta, on="district", validate="1:1")
print(gdf.drop(columns="geometry").round(2).head().to_string(index=False))
```

```
 district  tmean  mmt  rr_30C  af_pct
        0  17.05 18.0    1.09    6.32
        1  17.09 19.0    1.11    6.39
        2  17.45 19.1    1.12    6.06
        3  17.42 18.0    1.11    5.82
        4  17.84 18.9    1.13    5.61
```

Four points from this example matter in practice. First, `crossbasis` takes one exposure series at a time. A two-dimensional array is read as a matrix of exposure histories, one column per lag, so passing a (time, cell) array directly gives a wrong cross-basis without an error whenever the number of cells equals the number of lags plus one. The loop over districts avoids this. Second, the per-location outputs of `stack_reduced` and `blup` are arrays in the order the locations were supplied, so the analyst keeps the location key, as `meta` does here, and joins on it. Third, the MMT search needs the reduced coefficients paired with a `onebasis` of the exposure dimension and restricted to each location's own exposure range. Fourth, attribution from reduced coefficients uses the forward perspective, because reduced coefficients no longer carry the lag structure the backward perspective needs.

Treating small spatial units as locations needs care. Health outcomes are rarely recorded per grid cell, so outcomes are usually modelled at the unit where they are recorded, with gridded exposure averaged to that unit as above. Neighbouring units share weather and often population, so their first-stage estimates are correlated, which breaks the independence the meta-analysis assumes. Small units with few events make the first-stage estimates unstable. A spatially structured second stage is not implemented in dlnmpy, mixmetapy or the R packages they port, and small-area analyses may be better served by the case time series design (Gasparrini 2021), which is not yet ported. Because the objects dlnmpy returns can be pickled and the second stage needs only arrays, the first stage can be run in parallel with any task library. This paper has not tested that or measured its speed.

7 Relation to existing software

dlnm (Gasparrini 2011) remains the reference implementation and dlnmpy follows it. Where the two differ numerically, R's behaviour is the specification. dlnmpy is not affiliated with the

authors of dlnm. Users who need the parts of the R ecosystem that dlnmpy does not port, such as mgcv's internal `cb` smooths, the multilevel models in mixmeta or the case time series tools, should use R.

Two earlier Python efforts exist. pydlnm (Dessler, n.d.) covers the cross-basis, prediction, reduction and meta-analysis with BLUPs, reports its own validation against R, and delegates the spline bases and the model fit to R through rpy2, so it needs an R installation. The crossbasis package on PyPI builds the cross-basis only. dlnmpy differs from both in having no R dependency, in porting the attribution, MMT and penalised layers with R-compatible `pretty()`, `quantile()` and spline routines, with meta-analysis in mixmetapy, and in shipping the R-generated fixtures and a specification against which further ports can be checked.

8 Limitations

The package is at an alpha release. Its API, particularly the `dlnm()` workflow and the plotting functions, may still change.

The penalised fitter chooses smoothing parameters with a general-purpose optimiser on numerical derivatives, where mgcv uses a Newton method on analytic ones (Wood 2011). On well-conditioned models the two reach the same answer. On flat problems they can reach different smoothing parameters with almost the same score, as on one London model in Section 5.3. On the badly conditioned miners model the search is slow: in a run of `examples/biomet2017.py --all`, which the render does not include (its output is in `.handoff/biomet2017_all_output.txt`), it took 12 minutes on the machine described in Section 9.1 and reached a REML score 1.2×10^{-6} better than mgcv's, with smoothing parameters within 1.4% of mgcv's. Passing mgcv's smoothing parameters with `sp=` reproduces its coefficients. The log-determinant of the penalty in the criterion is computed directly from its eigenvalues, where mgcv uses the stable transformation of Wood (2011), so it loses accuracy when two smoothing parameters differ by a factor of about e^{20} or more. The selections reported here are far from that region. dlnmpy does not implement mgcv's UBRE criterion. Users who need mgcv's own smoothing-parameter selection should use R.

mixmetapy implements the single-level model of mixmeta. The multilevel and longitudinal extensions of Sera et al. (2019) are not ported, and Bayesian estimation is not offered.

dlnmpy is slower than R in places. In the render of this copy, on the machine described in Section 9.1, the one-call Chicago fit with its summary took 3 seconds and the attributable-risk table with 5,000 simulations 5 seconds. The translation of Gasparrini (2014) took 4 minutes, against 31 seconds for the R script that produced its reference results, which also refits each model to convergence. Most of the difference lies in `fit_clogit`, which evaluates the conditional likelihood one risk set at a time in Python through statsmodels, where `survival::coxph` uses compiled code, and in the `cr` basis, which evaluates its spline one value at a time in Python. Neither bottleneck changes a result.

The intervals the package reports are conditional, as are those of the R functions it ports. The empirical intervals for attributable numbers and for the MMT simulate the cross-basis coefficients from their estimated sampling distribution, so they carry coefficient uncertainty only. The dispersion, the confounder terms and the model specification are held fixed, and attributable numbers are computed relative to the point estimate of the MMT, as in the authors' code (Gasparrini et al. 2015). BLUP-based location estimates treat the between-location covariance as known, and their covariance is the formula of mixmeta, which dlnmpy reproduces and does not refine. Penalised fits report the Bayesian covariance conditional on the smoothing parameters, without mgcv's correction for their uncertainty (Wood 2017), and attribution from a penalised fit inherits this. The MMT and its interval are searched on a grid, of step 0.1 by default, within

the search range, so an interval that reaches the edge of the range is cut there. The simulated interval limits also carry Monte Carlo error, which is reported with the Chicago example in Section 4.

The checks are only as broad as the analyses compared. A combination of arguments that no fixture, side-by-side analysis or translation exercises has been checked only through the tests of its parts. As Section 5 sets out, most of the R code used for comparison came from the same development process as the port, and the translated published scripts are the only fully independent reference.

The package has one maintainer, and its continuous integration does not currently run. The port follows dlnm 2.4.10, the porting guide documents the mapping function by function, and the fixtures make divergence from R visible, but none of this replaces a second maintainer.

9 Reproducibility

The software described here is dlnmpy as it stands in the repository at the commit of this manuscript, version 0.8.5.dev0, with mixmetapy release 0.1.1. It is release 0.8.4 plus the fixes found by the statistical review for this paper (Section 5.5), none of which changes a number in the published analyses [CHECK: replace with release 0.8.5, its tag and its Zenodo DOI once released]. The repository also holds the source of this manuscript, the R scripts that produced the reference results, and the stored results themselves. `REPRODUCE.md` at the root of the repository gives the steps from a fresh clone.

In Python 3.11, from the repository root: `pip install -r paper/requirements.txt`, then `pip install --no-deps -e .` so that the document runs the code in the clone, then `quarto render paper/manuscript.qmd`. Rendering reruns the worked examples in Section 4, the side-by-side comparison, the six published translations and their comparisons with R in Section 5, and the gridded example in Section 6. `pytest` reruns the unit fixtures. Section 9.1 gives the times measured during the render of this copy.

`paper/requirements.txt` pins every Python package used for this manuscript and its dependencies, and `paper/.python-version` gives the Python version. Table 6 lists what was installed when this copy was rendered.

The Chicago data and the simulated cities ship with the package. The data for the six translations (England and Wales, London, the uranium miners and the climate projections) are downloaded on first render from pinned commits of the authors' repositories on GitHub and checked against SHA-256 hashes. They are not redistributed. Rendering therefore needs access to `raw.githubusercontent.com` once, after which the files are kept in `examples/` and, for the penalised DLNM paper, in `~/.cache/dlnmpy` (or the directory named by `DLNMPY_CACHE`). The gridded example is simulated in the document.

Every simulation takes a fixed seed: 0 for the MMT interval in `summary()` and the figure, 2026 for the Chicago attribution, 1000 plus the region index for the Lancet translation, and 1 for the simulated cities and the gridded example. The translations of Gasparrini and Leone (2014) and Vicedo-Cabrera, Sera, and Gasparrini (2019) replay the normal deviates and eigenvectors R used (seeds 20140421 and 13041975 plus the climate-model index on the R side), so their intervals are compared without Monte Carlo error. The library's own defaults are unseeded, so users who want repeatable intervals must pass a seed.

The render needs no R. The R sides of the comparisons are stored in `tests/reference/` and `tests/side_by_side/`, each with the versions that produced it (Table 6), and the scripts in `tools/` regenerate them from the authors' code. The R code of dlnm 2.4.7 and 2.4.10 dif-

fers only in `getvcov()` for `geeglm` models, which dlnmpy does not port. The fixture files in `tests/fixtures/` do not record the R versions that produced them.

9.1 Computational details

Table 6: Software used for this copy of the manuscript. The Python rows are read from the environment that rendered it. The R rows are read from the stored reference results, which were produced once with R and are not rerun.

Software	Version	Role
Python	3.11.15	interpreter
Quarto	1.6.43	rendering
dlnmpy	0.8.5.dev0	Python package
mixmetapy	0.1.1	Python package
numpy	2.4.4	Python package
scipy	1.17.1	Python package
pandas	3.0.2	Python package
statsmodels	0.15.0	Python package
matplotlib	3.10.9	Python package
xarray	2026.7.0	Python package
geopandas	1.1.4	Python package
shapely	2.1.2	Python package
formulaic	1.2.2	Python package
pytest	9.1.1	Python package
R	4.3.3	reference results only
dlnm	2.4.7	reference results only
mgcv	1.9.1	reference results only
mvmeta	1.0.3	reference results only
mixmeta	1.2.2	reference results only
tsModel	0.6.2	reference results only
survival	3.5.8	reference results only
MASS	7.3.60.0.1	reference results only

This copy was rendered on Linux x86_64, 4 logical CPUs. The references in Table 6 are R Core Team (2024), Gasparrini and Armstrong (2009), Gasparrini (2012), Sera and Gasparrini (2019), Wood (2017) with Wood (2000), Therneau and Grambsch (2000) with Therneau (1999), Venables and Ripley (2002) and Peng (2005) for R, and Harris et al. (2020), Virtanen et al. (2020), McKinney (2010) with The pandas development team (2020), Seabold and Perktold (2010), Hunter (2007), Hoyer and Hamman (2017), Jordahl et al. (2020) and Allaire et al. (2022) for Python. The six translations and the side-by-side comparison took 7 minutes of the render between them, of which the translation of Gasparrini (2014) took 4.

10 Declarations

Code availability. dlnmpy is at https://github.com/Logic06183/dlnmpy under GPL-2.0-or-later. The version described is 0.8.5.dev0, release 0.8.4 with the fixes listed in `CHANGELOG.md` [CHECK: release 0.8.5 and cite its version DOI here]. The latest release, 0.8.4 (Git tag `v0.8.4`), is archived on Zenodo with version DOI 10.5281/zenodo.23362540 (all versions: 10.5281/zenodo.22876735) and installable with `pip install dlnmpy==0.8.4`. mixmetapy is at https://github.com/Logic06183/mixmetapy under GPL-3.0-or-later, release 0.1.1 (Parker 2026b).

The source of this manuscript and the scripts it runs are in the dlnmpy repository under `paper/`, `examples/` and `tools/`, at the commit tagged [CHECK: tag of the submitted manuscript].

Data availability. All data are public or simulated. The sources, pinned commits and SHA-256 hashes are given in Section 9 and in each script in `examples/`.

Author contributions. CP: conceptualisation, methodology, software, validation, formal analysis, investigation, data curation, writing (original draft, review and editing), visualisation, project administration. [CHECK: CRediT roles of each co-author, and a description of any independent checks they made of the software or the manuscript.]

Use of AI. dlnmpy, mixmetapy and this manuscript were developed with substantial assistance from an AI coding assistant (Claude, Anthropic), over about seven weeks from late August 2026. The assistant drafted the Python code, the tests, the R scripts that produce the reference results, the translations of the published analyses, the audits, the documentation and the manuscript. Craig Parker designed the project, decided what to port and how it would be checked against R, directed the work, ran the comparisons and reviewed each defect fix and its test before release. Commits the assistant contributed to are marked as co-authored in the repository history. The checks described in Section 5 were produced by the same process, apart from the original authors' published R scripts they run. Craig Parker takes responsibility for the software and the analyses, and all authors for the manuscript.

Funding. This work was carried out within the HE²AT Center, supported by the US National Institutes of Health through the Data Science for Health Discovery and Innovation in Africa (DS-I Africa) programme, grant U54TW012083 [CHECK: grant number].

Competing interests. None.

Ethics. Not applicable. The study uses published, aggregated, publicly available data and simulated data.

11 Acknowledgements

The methods, R packages and scripts ported here are the work of others, and dlnmpy exists because they made their code and data public: Antonio Gasparrini, Ben Armstrong and Fabian Scheipl (dlnm), Michela Leone (`attrdl`), Aurelio Tobías (`findmin`), Francesco Sera (mixmeta), Ana Vicedo-Cabrera, Simon Wood (mgcv) and the R Core Team (splines and base). None of them has reviewed dlnmpy, and any errors are ours.

References

Allaire, J. J., Charles Teague, Carlos Scheidegger, Yihui Xie, and Christophe Dervieux. 2022. "Quarto." Zenodo. https://doi.org/10.5281/zenodo.5960048.

Dessler, Andrew. n.d. "pydlnm." GitHub repository. https://github.com/aedessler/pydlnm.

Gasparrini, Antonio. 2011. "Distributed Lag Linear and Non-Linear Models in R: The Package dlnm." *Journal of Statistical Software* 43 (8): 1–20. https://doi.org/10.18637/jss.v043.i08.

———. 2012. "mvmeta: Multivariate and Univariate Meta-Analysis and Meta-Regression." https://doi.org/10.32614/CRAN.package.mvmeta.

———. 2014. "Modeling Exposure-Lag-Response Associations with Distributed Lag Non-Linear Models." *Statistics in Medicine* 33 (5): 881–99. https://doi.org/10.1002/sim.5963.

———. 2021. "The Case Time Series Design." *Epidemiology* 32 (6): 829–37. https://doi.org/10.1097/EDE.0000000000001410.

Gasparrini, Antonio, and Ben Armstrong. 2009. "dlnm: Distributed Lag Non-Linear Models." https://doi.org/10.32614/CRAN.package.dlnm.

———. 2013. "Reducing and Meta-Analysing Estimates from Distributed Lag Non-Linear Models." *BMC Medical Research Methodology* 13: 1. https://doi.org/10.1186/1471-2288-13-1.

Gasparrini, Antonio, Ben Armstrong, and Michael G. Kenward. 2010. "Distributed Lag Non-Linear Models." *Statistics in Medicine* 29 (21): 2224–34. https://doi.org/10.1002/sim.3940.

———. 2012. "Multivariate Meta-Analysis for Non-Linear and Other Multi-Parameter Associations." *Statistics in Medicine* 31 (29): 3821–39. https://doi.org/10.1002/sim.5471.

Gasparrini, Antonio, Yuming Guo, Masahiro Hashizume, et al. 2015. "Mortality Risk Attributable to High and Low Ambient Temperature: A Multicountry Observational Study." *The Lancet* 386 (9991): 369–75. https://doi.org/10.1016/S0140-6736(14)62114-0.

Gasparrini, Antonio, and Michela Leone. 2014. "Attributable Risk from Distributed Lag Models." *BMC Medical Research Methodology* 14: 55. https://doi.org/10.1186/1471-2288-14-55.

Gasparrini, Antonio, Fabian Scheipl, Ben Armstrong, and Michael G. Kenward. 2017. "A Penalized Framework for Distributed Lag Non-Linear Models." *Biometrics* 73 (3): 938–48. https://doi.org/10.1111/biom.12645.

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. "Array Programming with NumPy." *Nature* 585: 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Hoyer, Stephan, and Joe Hamman. 2017. "xarray: N-D Labeled Arrays and Datasets in Python." *Journal of Open Research Software* 5 (1): 10. https://doi.org/10.5334/jors.148.

Hunter, John D. 2007. "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering* 9 (3): 90–95. https://doi.org/10.1109/MCSE.2007.55.

Jordahl, Kelsey, Joris Van den Bossche, Martin Fleischmann, et al. 2020. "geopandas." Zenodo. https://doi.org/10.5281/zenodo.3946761.

McKinney, Wes. 2010. "Data Structures for Statistical Computing in Python." In *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a.

Parker, Craig. 2026a. "dlnmpy: Distributed Lag Non-Linear Models in Python." Zenodo. https://doi.org/10.5281/zenodo.23362540.

———. 2026b. "mixmetapy: Multivariate Meta-Analysis and Meta-Regression in Python." Zenodo. https://doi.org/10.5281/zenodo.22876733.

Peng, Roger D. 2005. "tsModel: Time Series Modeling for Air Pollution and Health." https://doi.org/10.32614/CRAN.package.tsModel.

R Core Team. 2024. *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation for Statistical Computing. https://www.R-project.org/.

Seabold, Skipper, and Josef Perktold. 2010. "Statsmodels: Econometric and Statistical Modeling with Python." In *Proceedings of the 9th Python in Science Conference*, 92–96. https://doi.org/10.25080/Majora-92bf1922-011.

Sera, Francesco, Ben Armstrong, Marta Blangiardo, and Antonio Gasparrini. 2019. "An Extended Mixed-Effects Framework for Meta-Analysis." *Statistics in Medicine* 38 (29): 5429–44. https://doi.org/10.1002/sim.8362.

Sera, Francesco, and Antonio Gasparrini. 2019. "mixmeta: An Extended Mixed-Effects Framework for Meta-Analysis." https://doi.org/10.32614/CRAN.package.mixmeta.

The pandas development team. 2020. "pandas-dev/pandas: Pandas." Zenodo. https://doi.org/10.5281/zenodo.3509134.

Therneau, Terry M. 1999. "survival: Survival Analysis." https://doi.org/10.32614/CRAN.package.survival.

Therneau, Terry M., and Patricia M. Grambsch. 2000. *Modeling Survival Data: Extending the Cox Model*. New York: Springer. https://doi.org/10.1007/978-1-4757-3294-8.

Tobías, Aurelio, Ben Armstrong, and Antonio Gasparrini. 2017. "Investigating Uncertainty in the Minimum Mortality Temperature: Methods and Application to 52 Spanish Cities." *Epidemiology* 28 (1): 72–76. https://doi.org/10.1097/EDE.0000000000000567.

Venables, W. N., and B. D. Ripley. 2002. *Modern Applied Statistics with S*. 4th ed. New York: Springer. https://doi.org/10.1007/978-0-387-21706-2.

Vicedo-Cabrera, Ana M., Francesco Sera, and Antonio Gasparrini. 2019. "Hands-on Tutorial on a Modeling Framework for Projections of Climate Change Impacts on Health." *Epidemiology* 30 (3): 321–29. https://doi.org/10.1097/EDE.0000000000000982.

Virtanen, Pauli, Ralf Gommers, Travis E. Oliphant, et al. 2020. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." *Nature Methods* 17: 261–72. https://doi.org/10.1038/s41592-019-0686-2.

Wood, Simon N. 2000. "mgcv: Mixed GAM Computation Vehicle with Automatic Smoothness Estimation." https://doi.org/10.32614/CRAN.package.mgcv.

———. 2011. "Fast Stable Restricted Maximum Likelihood and Marginal Likelihood Estimation of Semiparametric Generalized Linear Models." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 73 (1): 3–36. https://doi.org/10.1111/j.1467-9868.2010.00749.x.

———. 2017. *Generalized Additive Models: An Introduction with R*. 2nd ed. Boca Raton, FL: Chapman; Hall/CRC. https://doi.org/10.1201/9781315370279.